

WORKIGOM FLOW — ZERNIO MULTI-TENANT SOCIAL INFRASTRUCTURE

MASTER IMPLEMENTATION PROMPT

0. GÖREVİN

Mevcut Workigom Flow uygulamasındaki sosyal medya entegrasyon mimarisini yeniden yapılandır.

Şu an sistemde Workigom'a ait veya geliştirici tarafından önceden oluşturulmuş bir Zernio Profile üzerinden sosyal hesap bağlantıları yapılıyor.

BU MİMARİ KALDIRILACAK.

Yeni mimaride her Workigom işletmesi kendi Zernio tenant/profile alanına sahip olacak ve kullanıcıların sosyal medya hesapları başka Workigom müşterileriyle kesinlikle karışmayacak.

Yeni kullanıcı deneyimi:

```
Google ile giriş
↓
Supabase Auth
↓
Workigom User
↓
Workigom Organization / Business
↓
Zernio Profile Provisioning
↓
Kullanıcı kendi sosyal hesaplarını bağlar
↓
Instagram / Facebook / LinkedIn / X / TikTok...
↓
Zernio Social Accounts
↓
Workigom account mapping
↓
Analytics / Publishing / Inbox / AI
```

Zernio kullanıcıya hiçbir yerde bir ürün/marka olarak gösterilmemelidir.

Zernio Workigom'un arka plandaki sosyal medya infrastructure provider'idir.

1. ÖNCE DISCOVERY YAP

KODLAMAYA HEMEN BAŞLAMA.

İlk olarak mevcut repository'yi incele.

Özellikle bul:

```
Supabase Google Auth
profiles
organizations
organization_members

social_accounts
posts
comments
reviews
conversations
messages

Zernio API client
Zernio API key kullanımı
Zernio profileId kullanımı
Zernio accountId kullanımı

social connect flow
OAuth callback route
analytics services
social media dashboard

Zernio webhook mevcut mu?
```

Şunları repository genelinde ara:

```
ZERNIO
profileId
accountId
/v1/connect
/v1/accounts
follower-stats
analytics
social_accounts
```

Mevcut Zernio entegrasyonunun tam dependency map'ini çıkar.

Raporla:

hangi dosyalar profileId kullanıyor?
hangi profileId hardcoded?
frontend Zernio API'ye doğrudan gidiyor mu?
API key client bundle'a sızıyor mu?
analytics nereden geliyor?
social account mapping nerede tutuluyor?

Discovery tamamlanmadan büyük refactor yapma.

2. ANA MİMARİ PRENSİP

Workigom kullanıcısı:

Zernio User

OLMAYACAK.

Authentication:

Google
↓
Supabase Auth
↓
Workigom

üzerinden gerçekleşecek.

Zernio authentication kullanıcıya gösterilmeyecek.

Workigom backend:

Workigom Organization
↓
Zernio Profile

eşleşmesini yönetecek.

3. USER DEĞİL ORGANIZATION MERKEZLİ OL

Zernio profile'ı doğrudan Supabase `auth.users.id` ile modelleme.

Çünkü aynı Workigom işletmesinde ileride:

işletme sahibi
çalışan
sosyal medya yöneticisi

gibi birden fazla kullanıcı bulunabilir.

Hepsi aynı şirketin sosyal hesaplarını kullanabilmelidir.

Canonical relation:

auth.users
↓
organization_members
↓
organizations
↓
integration.zernio_profiles

olmalıdır.

4. ZERNIO PROFILE MODELİ

Katı:

organization → ONE Zernio profile

tasarlama.

Model:

organization
↓
1..N Zernio profiles

desteklemelidir.

İlk onboarding sırasında bir adet:

primary profile

oluştur.

Çoğu işletmede yalnızca bu kullanılacaktır.

Ancak ileride:

```
aynı platformdan ikinci marka hesabı  
aynı organizasyon içinde farklı sosyal marka
```

gerektiğinde yeni profile oluşturulabilmelidir.

ÖNERİLEN TABLO:

```
integration.zernio_profiles  
  
id uuid primary key  
organization_id uuid not null  
zernio_profile_id text not null unique  
  
profile_key text not null  
display_name text  
  
is_primary boolean not null default false  
status text not null  
  
created_by uuid  
created_at timestamptz  
updated_at timestamptz
```

Status:

```
provisioning  
active  
error  
disabled
```

Unique:

```
organization_id + profile_key
```

olabilir.

Her organization yalnızca bir primary profile'a sahip olmalıdır.

Partial unique index kullan.

5. ZERNIO PROFILE PROVISIONING

Kullanıcı Google ile ilk kez giriş yaptıktan sonra:

```
Supabase Auth success
↓
Workigom organization oluştur/bul
↓
organization membership oluştur/bul
↓
Zernio profile mapping var mı?
```

kontrol et.

Varsa:

YENİ PROFILE OLUŞTURMA.

Yoksa server-side provisioning yap.

Zernio:

POST /v1/profiles

kullan.

Profile name insan tarafından yazılmış şirket adına bağlı olmamalıdır.

Bunu yapma:

Yılmaz Cafe

Bunun yerine deterministic internal identifier kullan:

wg_org_<organization_uuid>

veya güvenli kısa versiyonu.

Description:

Workigom organization: Yılmaz Cafe

olabilir.

6. PROFILE CREATE IDEMPOTENCY

Profile provisioning idempotent olmalıdır.

Zernio profile create isteğinde:

Idempotency-Key

kullan.

Örnek:

wg-zernio-profile:<organization_id>:primary

Aynı onboarding iki kere çalışırsa iki profile oluşmamalıdır.

Flow:

mapping ara
↓
bulundu → dön

bulunmadı
↓
createProfile
↓
mapping insert

Network timeout oluşursa request'i kör retry edip duplicate profile oluşturma.

Zernio 409 profile-name conflict döndürürse response'daki:

existingProfileId

bilgisini değerlendir ve deterministic name üzerinden recovery yap.

7. RACE CONDITION KORUMASI

İki browser tabı aynı anda onboarding çağırabilir.

Bu durumda:

iki Zernio profile

oluşmamalı.

Hem:

database UNIQUE constraint

hem:

Zernio Idempotency-Key

kullan.

Provisioning service concurrency-safe olmalıdır.

8. SERVER-ONLY ZERNIO CLIENT

Merkezi Zernio client oluşturun:

src/integrations/zernio/

Önerilen yapı:

```
zernio/
├── client.ts
├── config.ts
├── types.ts
├── errors.ts
├──
├── profiles/
│   └── profile.service.ts
├── accounts/
│   ├── account.service.ts
│   └── connect.service.ts
├──
├── analytics/
│   └── analytics.service.ts
├──
└── webhooks/
```

```
└─ webhook.service.ts
└─ webhook.types.ts
```

Mevcut repository conventions farklıysa mevcut mimariye adapte et.

Yeni paralel architecture yaratma.

9. API KEY GÜVENLİĞİ

Zernio API key:

```
browser
mobile app
React component
NEXT_PUBLIC_*
Expo public env
```

içine ASLA gitmeyecek.

Sadece:

```
Next.js server
Supabase Edge Function
secure backend
```

kullanabilir.

Örneğin:

```
ZERNIO_API_KEY
```

server-only environment variable.

Frontend:

```
Zernio
```

ile doğrudan konuşmayacak.

10. SOSYAL HESAP BAĞLAMA UX

Workigom Social Media ekranında:

Instagram	Bağla
Facebook	Bağla
LinkedIn	Bağla
X	Bağla
TikTok	Bağla
YouTube	Bağla

gibi seçenekler bulunacaktır.

User:

Instagram → Bağla

dediğinde:

```
Frontend
↓
Workigom API
↓
session doğrula
↓
organization belirle
↓
primary zernio profile belirle
↓
Zernio connect URL oluştur
↓
authUrl frontend'e dön
↓
browser OAuth'a gider
```

11. CONNECTION ENDPOINT

Önerilen Workigom endpoint:

POST /api/integrations/social/connect

Request:

```
{
  "platform": "instagram"
}
```

CLIENT:

```
organizationId
profileId
accountId
```

göndermemelidir.

Bunları server authenticated session'dan çözmelidir.

Backend:

```
user
↓
organization membership
↓
organization
↓
primary Zernio profile
```

bulur.

Ardından Zernio:

```
GET /v1/connect/{platform}
```

çağrılır.

Params:

```
profileId
redirect_url
```

12. STANDARD OAUTH İLE BAŞLA

İlk production versiyonunda:

```
headless=false
```

veya default Standard OAuth flow kullan.

Zernio selection UI'yi yönetsin.

Neden:

```
Facebook Pages  
LinkedIn organizations  
Instagram identities  
Pinterest boards  
Google Business locations  
WhatsApp numbers
```

gibi platform-specific seçim akışları vardır.

MVP'de bütün bu UI'yi Workigom içerisinde yeniden yazma.

13. HEADLESS FUTURE-READY OLSUN

Kod mimarisi ileride:

```
headless=true
```

desteklenebilecek şekilde olmalıdır.

Ancak şu sprintte custom selector ekranlarını üretme.

Service API:

```
createConnection({  
  organizationId,  
  platform,  
  mode: "standard"  
})
```

gibi abstraction kullanabilir.

Böylece ileride:

```
mode: headless
```

eklenebilir.

14. OAUTH CALLBACK

Workigom callback route oluřtur:

```
/api/integrations/zernio/callback
```

veya mevcut routing conventions'a uygun route.

Standard connection sonunda gelebilecek:

```
connected  
profileId  
accountId  
username
```

gibi params doęrulanmalıdır.

AMA:

Callback response tek source of truth kabul edilmemelidir.

Account mapping'in authoritative confirmation mekanizması webhook + reconciliation olmalıdır.

15. SOCIAL ACCOUNT DATABASE MODEL

Yeni canonical social account mapping tasarla.

Örneęin:

```
integration.social_accounts  
  
id uuid primary key  
  
organization_id uuid not null  
zernio_profile_mapping_id uuid not null  
  
zernio_profile_id text not null  
zernio_account_id text not null unique  
  
platform text not null
```

```
username text
display_name text
profile_picture_url text
profile_url text

is_active boolean
needs_reconnection boolean default false
enabled boolean default true

connected_at timestampz
disconnected_at timestampz
last_synced_at timestampz

metadata jsonb

created_at timestampz
updated_at timestampz
```

Ancak projede mevcut:

```
social_accounts
```

tablosu varsa duplicate yaratma.

Önce mevcut tabloyu incele.

Gerekirse migration ile genişlet.

16. PLATFORM ACCOUNT OWNERSHIP

En kritik güvenlik kurallarından biri:

Frontend'den gelen:

```
accountId
```

değerine güvenme.

Her social action öncesinde:

```
authenticated user
↓
organization
↓
social_accounts
```

↓
zernio_account_id

ownership doğrulanmalıdır.

Örnek:

```
assertSocialAccountOwnership({  
  organizationId,  
  zernioAccountId  
})
```

başarısızsa:

403

dön.

17. ACCOUNT SYNC

Zernio:

GET /v1/accounts?profileId=...

ile profil altındaki hesapları listeleyebilir.

`syncZernioAccounts()` servisi oluştur.

Input:

organizationId
zernioProfileId

Flow:

```
Zernio accounts getir  
↓  
her account için ownership/profile check  
↓  
upsert local mapping
```

↓
eksik/removed accountları reconcile et

UI her sayfa açıldığında Zernio API çağtırmamalıdır.

UI normalde Supabase local projection'dan okumalıdır.

18. WEBHOOK ENDPOINT

Tek production endpoint:

POST /api/webhooks/zernio

oluştur.

Webhook event'i profileId üzerinden tenant'a route et.

Minimum destek:

account.connected
account.disconnected

Sonraki aşamalarda:

post.published
post.failed

message.received
conversation.started

comment.received
review.new

eklenebilir.

19. WEBHOOK SECURITY

Zernio webhook HMAC verification destekliyorsa:

X-Zernio-Signature

doğrula.

Webhook secret server secret olarak tutulmalıdır.

Signature başarısızsa:

```
401 / 403
```

ver.

Event body işlenmeden önce signature doğrulanmalıdır.

20. WEBHOOK EVENT DEDUPLICATION

Webhook payload içindeki stable:

```
id
```

event ID olarak kullanılmalıdır.

Yeni tablo veya mevcut event infrastructure:

```
integration.webhook_events
```

Örnek:

```
provider
external_event_id
event_type
received_at
processed_at
status
error
```

Unique:

```
provider + external_event_id
```

aynı webhook retry edildiğinde ikinci kez side effect üretmesini engellesin.

21. ACCOUNT.CONNECTED

Event:

```
account.connected
```

geldiğinde payload'dan:

```
accountId  
profileId  
platform  
username  
displayName
```

al.

Flow:

```
profileId  
↓  
integration.zernio_profiles  
↓  
organization_id
```

bul.

Sonra:

```
social account upsert
```

yap.

Profile mapping bulunamıyorsa sessizce yanlış organization'a yazma.

Event:

```
orphaned
```

olarak loglanabilir ve alert üretilebilir.

22. ACCOUNT.DISCONNECTED

Event:

```
account.disconnect
```

geldiğinde local social account silinmemelidir.

Şöyle işaretle:

```
is_active = false  
needs_reconnection = true  
disconnected_at = timestamp
```

Payload'taki:

```
intentional  
unintentional  
reason
```

değerlerini metadata/event history içinde sakla.

UI:

```
Instagram  
@firma  
  
Bağlantı yenilenmeli  
  
[Yeniden Bağla]
```

gösterebilmelidir.

23. RECONNECT FLOW

Reconnect yeni Workigom organization veya yeni primary profile oluşturmamalıdır.

Mevcut:

```
zernio_profile_id
```

kullanılarak connect flow tekrar başlatılmalıdır.

24. MULTIPLE ZERNIO PROFILES

Her işletme varsayılan olarak yalnızca:

```
primary
```

profile kullanacaktır.

Ancak profile service:

```
getPrimaryProfile()  
createProfileSlot()  
listProfiles()
```

gibi future-safe API sağlayabilir.

Özellikle aynı organization'ın aynı platformdan ikinci account bağlama ihtiyacı oluştuğunda:

```
secondary profile
```

oluşturulabilmelidir.

BU sprintte kullanıcıya karmaşık multi-profile UI gösterme.

Backend capability hazır olsun.

25. INSTAGRAM ÖZEL KISIT

Instagram Facebook Login selection flow'unda aynı Zernio profile üzerinde mevcut Instagram hesabını başka Instagram hesabıyla değiştirmek eski identity'nin:

```
conversations  
external posts  
stats
```

verilerini temizleyebilir.

Bu yüzden kullanıcı mevcut profile üzerinde başka Instagram bağlamak isterse körlemesine replacement yapma.

Policy oluştur:

Profile üzerinde instagram var mı?

↓

HAYIR → bağla

EVET

↓

Yeni ayrı Zernio profile oluştur

VEYA

kullanıcı açıkça replace istiyorsa confirmation

Default davranış:

data destructive replacement YAPMA.

26. ANALYTICS ARCHITECTURE

Analytics doğrudan React component içinde Zernio'dan çekilmemelidir.

Katman:

Flow UI

↓

Analytics Service

↓

Local Cache / Projection

↓

Zernio Analytics API

olmalı.

27. FOLLOWER DATA

Zernio SocialAccount response'unda uygun plan/add-on varsa:

followersCount
followersLastUpdated

bulunabilir.

Ayrıca follower stats endpoint'i kullanılıyorsa bunu server-side service'e wrap et.

Örnek:

```
getFollowerStats({
  organizationId,
  socialAccountId,
  from,
  to,
  granularity
})
```

UI:

zernioAccountId

ile doğrudan API çağırılmamalıdır.

28. ANALYTICS CAPABILITY MODEL

Platformların analytics kabiliyetleri birbirinden farklıdır.

Tek tip sahte veri modeli dayatma.

Örneğin:

```
interface SocialCapabilities {
  followers: boolean
  impressions: boolean
  reach: boolean
  engagement: boolean
  comments: boolean
  postAnalytics: boolean
}
```

gibi capability layer oluşturun.

UI'da platform desteklemeyen metric:

0

gösterilmemelidir.

Bunun yerine:

–
Bu platform için mevcut değil

gibi state kullanılmalıdır.

29. LOCAL ANALYTICS CACHE

Mevcut analytics storage varsa kullan.

Yoksa şu modele benzer düşün:

```
flow.social_account_metrics
```

veya mevcut DDD schema kararına göre uygun tablo.

Minimum:

```
organization_id  
social_account_id  
metric_date  
followers  
impressions  
reach  
engagements  
posts_count  
raw_metrics  
synced_at
```

Her platformdaki özel metrikler:

```
raw_metrics JSONB
```

içinde tutulabilir.

Normalized alanlar ortak Workigom dashboard'ı için kullanılır.

30. BACKGROUND SYNC

Her page load'da Zernio analytics çekme.

Background sync kullan.

Örneğin:

```
Account bağlandı
↓
initial sync

sonra
↓
scheduled controlled refresh
```

Current project infrastructure içinde cron/queue varsa kullan.

Yoksa bu sprintte ağır queue sistemi kurmadan basit scheduled sync tasarla.

API rate limits'e uygun davran.

31. SOCIAL MEDIA SCREEN

Mevcut Workigom Social Media ekranının fonksiyonları korunacak.

Yeni data source:

```
current developer-owned Zernio profile
```

yerine:

```
authenticated organization
↓
integration.zernio_profiles
↓
integration.social_accounts
```

olacaktır.

32. ACCOUNT CARDS

Bağlı hesap kartında:

```
platform
profile photo
display name
username
```

```
connection status
last sync
```

göster.

Actions:

```
Yönet
Senkronize Et
Yeniden Bağla
Kaldır
```

duruma göre görünür.

33. USER ONBOARDING

Google login sonrası business yoksa:

Step 1

```
Workigom'a hoş geldiniz

İşletmenizi oluşturalım.

İşletme adı
...
```

Organization oluştur.

Step 2

Backend otomatik:

```
Zernio primary profile provisioning
```

yapar.

Kullanıcıya:

```
Zernio profili oluşturuluyor
```

deme.

UX:

İşletmeniz hazırlanıyor...

olabilir.

Step 3

Sosyal hesaplarınızı bağlayın

Instagram	Bağla
Facebook	Bağla
LinkedIn	Bağla
X	Bağla
TikTok	Bağla

Şimdilik Geç

34. EXISTING USERS MIGRATION

Mevcut Workigom kullanıcıları şu anda ortak/developer Zernio profile kullanıyor olabilir.

BU DURUM ÇOK DİKKATLİ MIGRATE EDİLMELİ.

Önce rapor çıkar:

```
hangi Workigom organization
hangi mevcut Zernio accountId
hangi mevcut profileId
```

mapping'i var?

Mevcut sosyal hesapları otomatik başka profile taşımanın Zernio tarafından güvenli desteklenip desteklenmediğini OpenAPI'den doğrula.

Doküman açıkça desteklemiyorsa:

hesabı sessizce taşıma.

Existing users için kontrollü:

Sosyal hesabınızı yeniden bağlayın

akışı gerekebilir.

Veri kaybına yol açabilecek migration yapma.

35. LEGACY PROFILE'IN KULLANIMINI DURDUR

Yeni kullanıcılar hiçbir koşulda mevcut developer/shared profile'a bağlanmamalıdır.

Feature flag veya migration aşamasında:

ZERNIO_MULTI_TENANT_PROVISIONING=true

benzeri kontrollü rollout kullanılabilir.

Yeni user:

always organization profile

Existing user:

legacy compatibility until migrated

şeklinde geçici davranabilir.

36. API ROUTES

Önerilen backend surface:

POST
/api/integrations/social/profile/ensure

GET
/api/integrations/social/accounts

POST
/api/integrations/social/connect

GET
/api/integrations/zernio/callback

```
POST
/api/integrations/social/sync

POST
/api/integrations/social/accounts/:id/reconnect

DELETE
/api/integrations/social/accounts/:id

POST
/api/webhooks/zernio
```

Ancak mevcut Next.js architecture Server Actions kullanıyorsa uygun olanları actions/services katmanına adapte et.

HTTP endpoint yaratmak için endpoint yaratma.

OAuth ve webhook için route handler mantıklıdır.

37. REPOSITORY LAYER

UI aşağıdakileri bilmemelidir:

```
Zernio profileId
raw Zernio schema
Zernio API URL
API key
```

Repository/service API:

```
SocialIntegrationService.ensureBusinessProfile()

SocialIntegrationService.getConnectedAccounts()

SocialIntegrationService.connectAccount()

SocialIntegrationService.disconnectAccount()

SocialIntegrationService.reconnectAccount()

SocialAnalyticsService.getDashboardMetrics()
```

şeklinde olmalıdır.

38. AUTHORIZATION

Her route/server action:

```
supabase.auth.getUser()
```

ile authenticated user'ı server-side çözmelidir.

Ardından:

```
organization_members
```

üzerinden organization erişimini doğrula.

Client'tan gönderilen:

```
organizationId  
zernioProfileId  
zernioAccountId
```

değerlerini authorization olmadan kullanma.

39. RLS

Yeni/taşınan Supabase tablolarında RLS oluştur.

Workigom kullanıcıları yalnızca üyesi oldukları organization'ın:

```
zernio profile mapping  
social account mapping  
analytics
```

kayıtlarını görebilmelidir.

Service role kullanan webhook/backend operasyonlarında da tenant routing explicit olmalıdır.

40. ERROR MODEL

Normalize edilmiş error modeli oluştur:

```
type SocialIntegrationErrorCode =  
  | "PROFILE_PROVISION_FAILED"  
  | "PROFILE_LIMIT_REACHED"  
  | "ACCOUNT_CONNECTION_FAILED"  
  | "ACCOUNT_NOT_FOUND"  
  | "ACCOUNT_ACCESS_DENIED"  
  | "ACCOUNT_RECONNECT_REQUIRED"  
  | "ANALYTICS_NOT_AVAILABLE"  
  | "ZERNIO_UNAVAILABLE"
```

Zernio raw error mesajlarını kullanıcıya doğrudan gösterme.

41. RETRY STRATEGY

GET/read işlemleri kontrollü retry olabilir.

Profile create:

```
Idempotency-Key
```

ile retry.

Write/external operations:

API'nin idempotency garantisi doğrulanmadan blind retry yapma.

Exponential backoff:

```
429  
502  
503  
504
```

gibi retryable durumlarda düşünülebilir.

42. OBSERVABILITY

Her integration operation için:

```
organization_id  
operation  
provider = zernio
```

```
profile mapping id
social account id
platform
status
latency
error_code
timestamp
```

loglanmalı.

Secret/token loglama.

OAuth tempToken/access token kesinlikle normal application logs içine yazılmamalıdır.

43. HEALTH / RECONCILIATION

Webhook tek başına yeterli kabul edilmemelidir.

Periyodik reconciliation:

```
local active accounts
vs
Zernio GET /v1/accounts?profileId
```

karşılaştırabilir.

Ama bu yüksek frekanslı polling olmamalıdır.

Amaç:

```
kaçırılan webhook
stale connection status
```

gibi durumları düzeltmek.

44. TESTLER

Provisioning

```
Yeni Google user
→ organization
```

```
→ primary Zernio profile
→ mapping
```

oluşmalı.

İki kez çağrılırsa:

```
tek profile
```

kalmalı.

Tenant isolation

Org A'nın user'ı:

```
Org B zernio accountId
```

ile işlem yapmaya çalışırsa:

```
403
```

olmalı.

Connect

Instagram connect:

```
correct organization
correct Zernio profile
correct redirect
```

kullanmalı.

Webhook

```
account.connected :
```

```
profileId
↓
correct organization
```

↓
account upsert

yapmalı.

Duplicate event:

ikinci side effect üretmemeli.

Disconnect

account.disconnected :

account silinmemeli
needs_reconnection = true

olmalı.

Reconnect

Reconnect:

aynı organization/profile

üzerinde çalışmalı.

Cross-tenant

Org A frontend manipülasyonla:

Org B accountId

gönderirse reddedilmeli.

Analytics

Analytics isteyen kullanıcı yalnızca kendi organization account'larının metrics'ini alabilmeli.

Analytics capability yoksa:

supported:false

veya düzgün empty state dönmeli.

Sahte ☐ üretme.

45. UX ACCEPTANCE TEST

Yeni kullanıcı deneyimi:

- 1 Google ile giriş
- 2 İşletme oluştur
- 3 İşletme hazırlanıyor
- 4 Sosyal hesaplarınızı bağlayın
- 5 Instagram → Bağla
- 6 OAuth
- 7 Workigom'a geri dön
- 8 Instagram hesabı bağlı görünür
- 9 Dashboard follower bilgisi gösterir
- 10 Kullanıcı gönderi oluşturabilir
- 11 Gelen kutusu yalnızca kendi hesaplarını gösterir
- 12 Analytics yalnızca kendi hesaplarını gösterir

Bu akışta kullanıcı:

Zernio workspace
Zernio profile
Zernio API key

görmemelidir.

46. MEVCUT FLOW FONKSİYONLARINI KIRMA

Mobil ve web aynı backend'i kullanıyorsa:

```
Mobile Flow
Web Flow
```

aynı SocialIntegrationService ve aynı canonical database mapping'i kullanmalıdır.

Web'e ayrı Zernio logic, mobile'a ayrı Zernio logic

YAZMA.

Tek backend contract.

İki presentation layer.

47. DATABASE DOMAIN ARCHITECTURE İLE UYUMLU OL

Daha önce kararlaştırılan DDD mimarisiyle çalışma.

Zernio infrastructure:

```
integration
```

domain'inde olmalıdır.

Social product/business data:

```
flow
```

veya mevcut canonical sosyal domain'de tutulmalıdır.

Identity:

```
core
```

olmalıdır.

Örnek:

```
core.organizations
```

```
integration.zernio_profiles
```

```
integration.social_accounts
```

```
integration.webhook_events
```

```
flow.social_account_metrics
```

```
flow.posts
```

Ancak mevcut tablolar varsa duplicate oluşturma.

48. ÖNERİLEN EXECUTION PLAN

PHASE 1 — DISCOVERY

Mevcut Zernio architecture dependency report.

KOD YOK.

PHASE 2 — DATABASE MAPPING

Canonical tabloları belirle.

Migration plan çıkar.

PHASE 3 — ZERNIO SERVER CLIENT

Secure client + profile service.

PHASE 4 — PROFILE PROVISIONING

Organization → Zernio Profile.

PHASE 5 — CONNECTION FLOW

Connect URL + callback.

PHASE 6 — ACCOUNT MAPPING

Local social account projection.

PHASE 7 — WEBHOOK

Connected/disconnected + deduplication + HMAC.

PHASE 8 — ANALYTICS

Follower / account / post analytics abstraction.

PHASE 9 — UI CONNECTION

Flow Social Media screens use new tenant-aware backend.

PHASE 10 — LEGACY MIGRATION

Shared Zernio profile kullanımını kaldır.

49. DESIGN REVIEW GATE

Her phase sonunda DUR.

Şunları raporla:

ne değişti?
hangi dosyalar değişti?
hangi migration uygulandı?
hangi testler geçti?
hangi risk kaldı?

Bir phase çalışmadan diğerine geçme.

50. İLK TESLİMAT

İlk implementasyon teslimatında şu senaryo production-quality çalışmalıdır:

```
Yeni kullanıcı
↓
Google Login
↓
Workigom Organization
↓
Automatic Zernio Profile
↓
Instagram Connect
↓
Zernio OAuth
↓
account.connected webhook
↓
Local social account mapping
↓
Workigom Social Media UI
↓
Instagram hesabı görünür
```

Ardından:

```
followers / account metrics
```

kullanıcının KENDİ hesabından alınmalıdır.

Başka Workigom organization'ın hiçbir Zernio hesabı görünmemelidir.

51. EN KRİTİK KURAL

Asla tekrar şu mimariye dönme:

```
WORKIGOM USERS
↓
ONE SHARED DEVELOPER ZERNIO PROFILE
↓
ALL SOCIAL ACCOUNTS
```

Doğru yapı:

```
WORKIGOM USER
↓
MEMBERSHIP
↓
```

WORKIGOM ORGANIZATION



ZERNIO PROFILE MAPPING



THAT ORGANIZATION'S SOCIAL ACCOUNTS

Ve gerektiğinde:

ONE ORGANIZATION



MULTIPLE ZERNIO PROFILES

desteklenebilmelidir.

52. ŞİMDİ YAPMAN GEREKEN

İlk olarak yalnızca **PHASE 1 — DISCOVERY** gerçekleştir.

Henüz migration çalıştırma.

Henüz mevcut Zernio profile'ını silme.

Henüz mevcut kullanıcıların sosyal hesap bağlantılarını değiştirme.

Bana şu başlıklarda rapor ver:

1. Current Google/Supabase onboarding flow
2. Current organization creation flow
3. Current Zernio profile architecture
4. Hardcoded/shared profile usages
5. Current Zernio API client locations
6. Current social account table/schema
7. Current OAuth connection flow
8. Current analytics flow
9. Current webhook infrastructure
10. Mobile + Web dependencies
11. Security problems
12. Proposed migration files
13. Files that must change
14. Legacy account migration risk
15. Final implementation plan

Discovery raporunu onaylamadan production verisini değiştirme.